

# **Práctica 4:**

## **Filtro SMA de orden 4**

Diseño automático de sistemas fiables

Victor Arroyo Márquez

Jose Fernando Marcilla Morán

Marcelo Fort Muñoz

{mfortm,varroyom,jmarcillam1}@alumnos.nebrija.es

## Índice

|                                                                            |   |
|----------------------------------------------------------------------------|---|
| Práctica 4: Filtro SMA de orden 4.....                                     | 1 |
| 1. Introducción y objetivos.....                                           | 3 |
| 2. Descripción del diseño.....                                             | 3 |
| 3. Verificación.....                                                       | 4 |
| A. Triangular y reset inicial.....                                         | 4 |
| B. Cuadrada.....                                                           | 4 |
| C. Dientes de sierra.....                                                  | 4 |
| D. Comprobaciones automáticas por código.....                              | 4 |
| 4. Análisis de resultados.....                                             | 5 |
| A. Sobre la suavidad de la entrada.....                                    | 6 |
| B. Sobre el retardo respecto a la señal original.....                      | 7 |
| C. Sobre la respuesta de la máquina al cambio repentino de la entrada..... | 7 |
| D. Qué señal muestra más claramente el filtrado.....                       | 8 |
| 5. Conclusiones.....                                                       | 8 |

## 1. Introducción y objetivos

Cuando se obtienen muchos datos de un sensor físico, a veces hay datos erróneos o variaciones muy bruscas que pueden debilitar la calidad de la información adquirida. Para esto, se puede hacer medias de los datos. Sin embargo, con hacer la media no es suficiente. Por eso, hacemos la media móvil, qué va «cogiendo» los datos según van entrando y, devuelve las media entre los  $N$  últimos números, suavizando la salida.

En este caso, con  $N = 4$ , el filtro sólo carga nuevos datos cuando `load='1'`. Además, para dividir entre 4 se realiza un desplazamiento de dos bits a la derecha, ahorrando recursos de la FPGA destino.

En nuestro caso, nuestra máquina es síncrona (es decir, funciona en base a un reloj) y, de orden 4 (es decir, funciona con cuatro muestras para la media). Los datos son de 8 bits sin signo.

## 2. Descripción del diseño

Se ha diseñado un componente que, como especificado en el enunciado, cumple con la siguiente interfaz:

```
entity sma_filter is
    Port ( clk : in STD_LOGIC;
          rst : in STD_LOGIC;
          din : in STD_LOGIC_VECTOR (7 downto 0);
          load : in STD_LOGIC;
          dout : out STD_LOGIC_VECTOR (7 downto 0));
end sma_filter;
```

Esta tiene:

- Un reloj.
- Una entrada de 8 bits.
- Un bit selector de carga, una especie de «load enable» .
- Una salida de 8 bits.
- 4 registros internos de 8 bits en los que se almacenan las entradas para hacer la media móvil.

Cuando load está activada, se van sumando los registros cada ciclo y se hace un desplazamiento, haciendo una división efectiva entre 4.

### 3. Verificación

Siendo esta práctica un recorrido por todo el proceso de diseño y verificación, no se podría justificar la ausencia de esta sección. Para comprobar que el código funcionaba correctamente, hemos probado con múltiples estímulos.

A continuación detallamos lo implementado en la verificación.

#### A. Triangular y reset inicial.

Primero, reseteamos el sistema para evitar valores indeterminados o incorrectos en la máquina. Esto lo hacemos poniendo a valor alto el puerto de reset, esperando un ciclo y, posteriormente, poniéndolo de nuevo a valor bajo.

Luego, introducimos unas entradas que simulan la forma de triángulo, es decir, incrementan con un factor estable para luego descender con ese mismo factor, haciendo la forma de triángulo con dos «rampas», una de «subida» y otra de «bajada». Estos estímulos nos permiten ver como la máquina responde a una serie de estímulos más normales y predecibles.

#### B. Cuadrada

A continuación, se implementó una serie de estímulos que cumplían el siguiente patrón: mantienen el valor mínimo, saltan al máximo y se mantienen en el durante un par de ciclos para a continuación volver al mínimo. Estos estímulos nos permiten ver como nuestro diseño responde ante un caso de variabilidad extrema.

#### C. Dientes de sierra

Finalmente, probamos con un patrón de dientes de sierra. Este se caracteriza por ser asimétrico y con cambios repetitivos. Permite ver muy bien cómo responde el autómata ante una entrada con una especie de desvío o entrada anómala.

#### D. Comprobaciones automáticas por código

Hemos automatizado ciertas pruebas usando la suite que VHDL nos proporciona.

En nuestro caso, hemos comprobado:

- I. Que tras la entrada cuatro veces del valor máximo, la salida se corresponde con este (línea 150).
- II. Que tras la activación del bit de reset la salida se ponga a 0 (línea 213).
- III. Que después del reset el sistema funciona correctamente (línea 228)
- IV. Que después de desactivar el bit de habilitar carga (load), el sistema se «congela», es decir, mantiene su estado y no recoge los datos de la entrada.

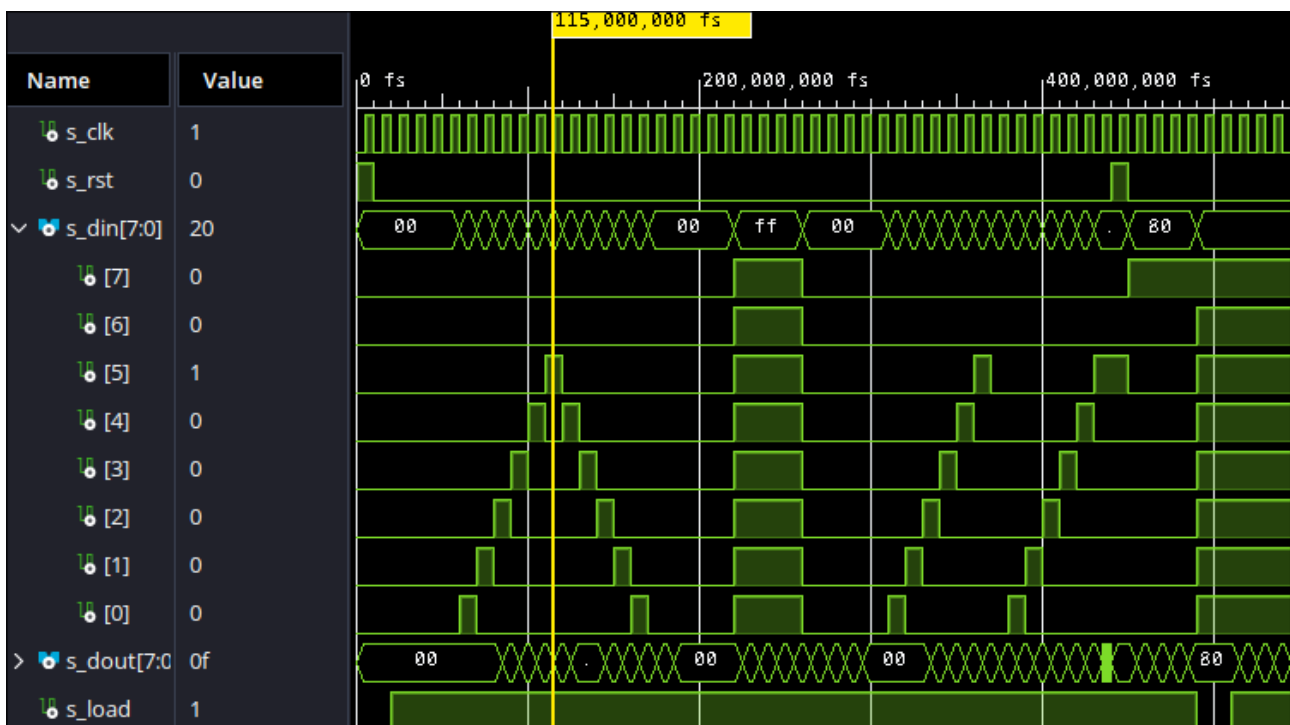


Figure 1: Ejecución de la simulación con estímulos descritos en las secciones 3.A, 3.B y 3.C

Nótese que no se muestran los contenidos de `s_dout`. Esto es intencionado, ya que se estudiarán en la sección venidera y penúltima a este informe de laboratorio.

```
Time resolution is 1 ps
Note: --- INICIO DEL TESTBENCH ---
Time: 20 ns Iteration: 0 Process: /tb_media/line__64 File: /home/marcelo/Document
Note: Exito: El reset hace lo que promete
Time: 445 ns Iteration: 0 Process: /tb_media/line__64 File: /home/marcelo/Document
Note: Si se llenan los cuatro registros, la media da el valor esperado y adecuado
Time: 490 ns Iteration: 0 Process: /tb_media/line__64 File: /home/marcelo/Document
Note: Load funciona
Time: 510 ns Iteration: 0 Process: /tb_media/line__64 File: /home/marcelo/Document
Note: Todo funciona
Time: 510 ns Iteration: 0 Process: /tb_media/line__64 File: /home/marcelo/Document
```

Figure 2: Log después de una simulación acorde a lo escrito en el apartado 3.D

## 4. Análisis de resultados

Procedemos ahora a la fase de análisis de resultados. Igual que en la anterior, justificaremos esta implementación que hemos realizado con datos comprobables y reales.

Con permiso del enunciado y justificando la escasez de este apartado, nos permitiremos la licencia de contestar solamente a las preguntas que han sido formuladas en las guías de la práctica.

### A. Sobre la suavidad de la entrada

Se pregunta en el enunciado si nuestra salida es más suave que la entrada. Nuestra respuesta a esta pregunta es sí, más es una suavidad algo difícil de apreciar al ser el orden 4.

En la Figure 3 podemos ver cómo en el caso de la entrada triangular, con un retardo, los valores que van saliendo con la media móvil crecen de una forma más equilibrada que la entrada. En la Figure 4 vemos como en lugar de saltar a 255 directamente cómo en la entrada, pasa por 63, 127 y 191 antes, suavizando esa entrada. Posteriormente, repite ese patrón a la inversa, suavizando también la bajada al contrario que la entrada, que salta a 0. Finalmente, en el caso de la sierra, vemos en la Figure 5 como también se suaviza la entrada respecto a la entrada original.

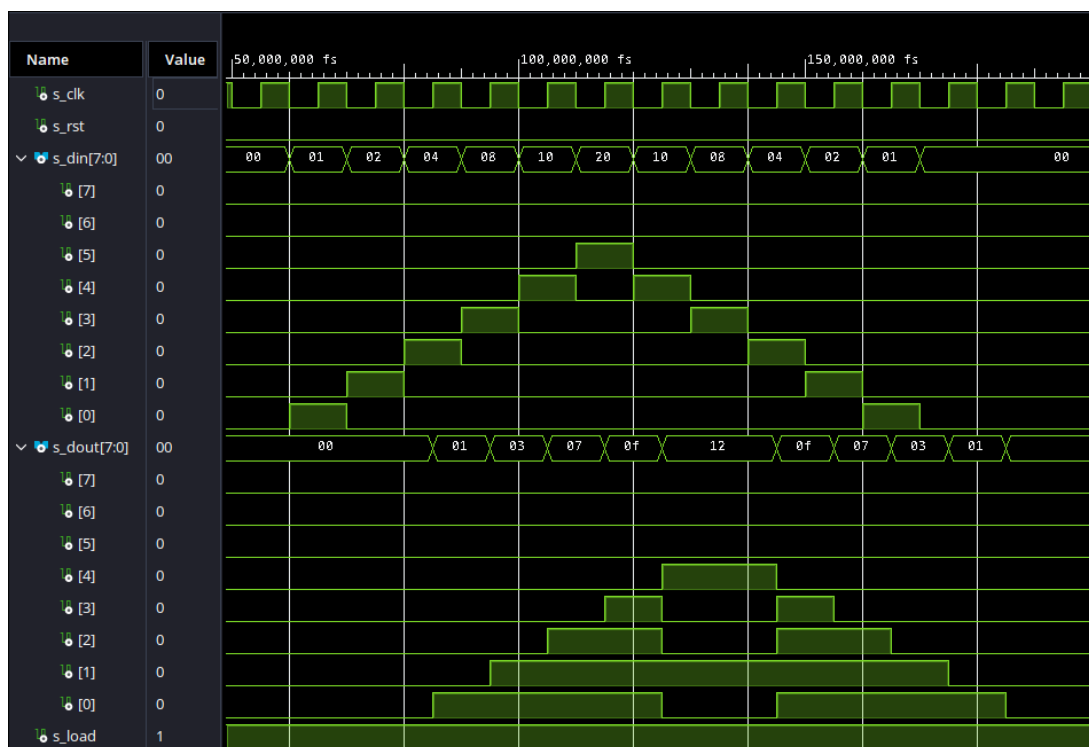


Figure 3: Comparación de salidas de triangular: entrada vs salida

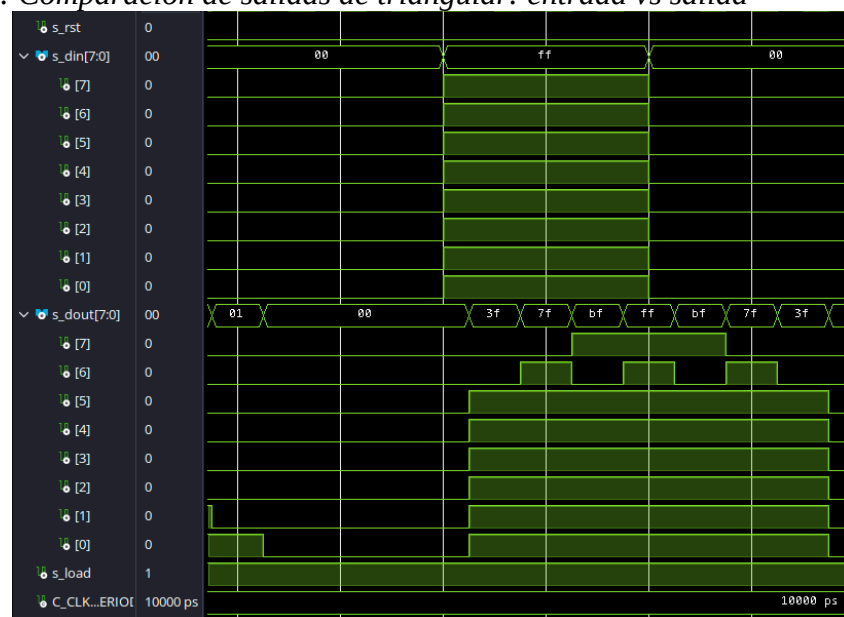


Figure 4: Comparación de salidas de cuadrada: entrada vs salida

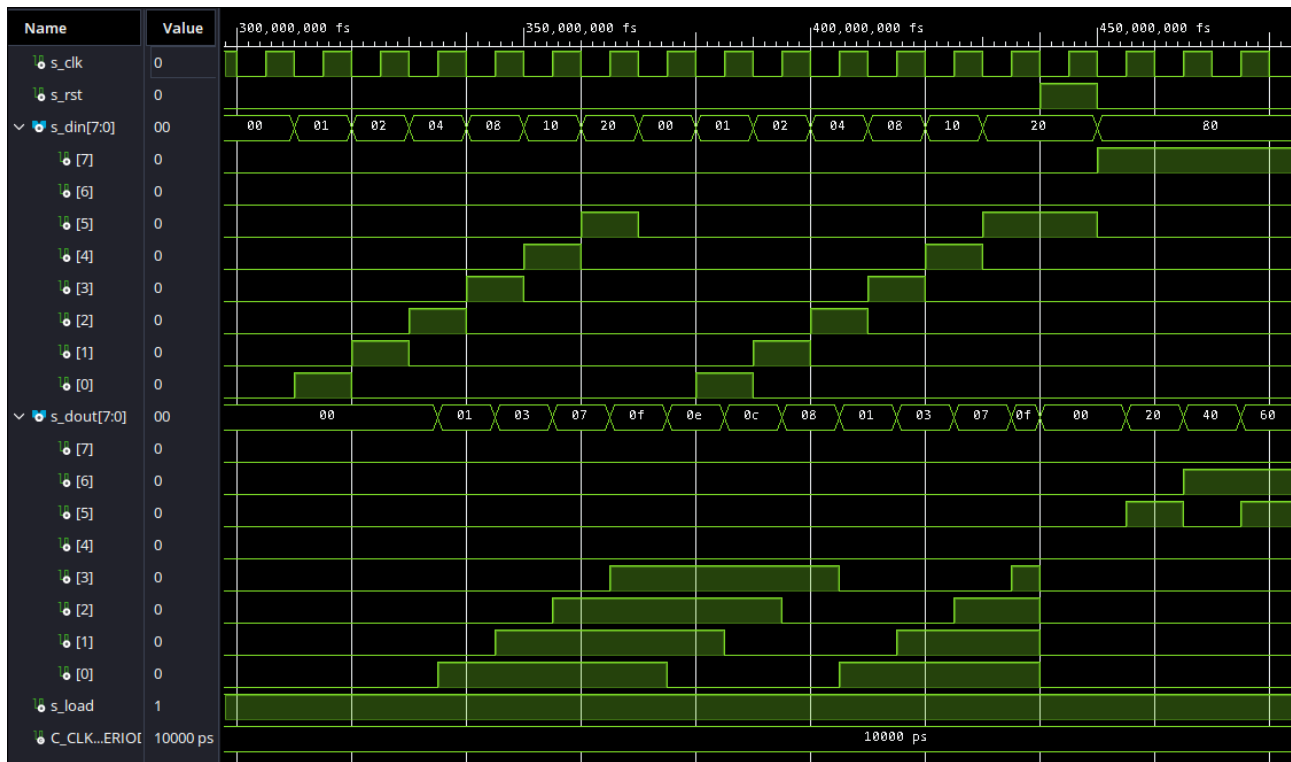


Figure 5: Comparación de salidas de sierra: entrada vs salida

### B. Sobre el retardo respecto a la señal original

Efectivamente, sí existe este retardo. Esto se debe a que la entrada depende de las cuatro muestras para actualizarse y, por ello, tiene una cierta latencia. Este efecto se ve con claridad en las figuras Figure 3, Figure 4 y Figure 5.

### C. Sobre la respuesta de la máquina al cambio repentino de la entrada

Cuando se recibe una entrada con un salto instantáneo, la salida va adquiriendo ese cambio de forma amortiguada por la media móvil. Al ser el filtro de cuarto orden, este requiere de que los nuevos valores vayan llenando su memoria con los nuevos valores, escalonando esa transición progresiva. Por ejemplo, al cambiar de 0 a 255, la salida va tomando los valores 63, 127 y 191 antes de llegar a una convergencia final al valor máximo (que dicho de otra forma, es 255). Este comportamiento es el que se espera con un SMA. Sin cambios súbitos, siempre aumentos escalonados.

#### **D. Qué señal muestra más claramente el filtrado.**

Claramente, cómo se observa en la Figure 4, el patrón «favorito» para ver el filtrado es el cuadrado. Esto se debe a que debido al gran salto se puede ver el escalado de la media móvil contra el cambio brusco que se da en la entrada. Las transiciones son suaves en comparación y hay un retardo que justifica el apartado anterior.

### **5. Conclusiones**

Tras el desarrollo y verificación del filtro SMA de orden 4, podemos extraer las siguientes conclusiones:

- **Cumplimiento de Objetivos:** Se ha implementado con éxito un módulo VHDL sintetizable capaz de procesar señales de 8 bits mediante una ventana deslizante. El diseño respeta íntegramente la arquitectura síncrona.
- **Validación del Filtrado:** Mediante la simulación de señales triangulares, cuadradas y de sierra, se ha validado cualitativamente el efecto de suavizado y amortiguación de transiciones bruscas. La respuesta del filtro ante la señal cuadrada ilustra con claridad el retardo de 4 muestras que introduce el procesado.
- **Robustez de Verificación:** El uso de un testbench exhaustivo con comprobaciones automáticas mediante assert ha permitido confirmar la correcta inicialización del sistema tras el reset, la convergencia de la media y la estabilidad del estado interno ante el control de carga (load).

En definitiva, la práctica ha servido para asentar los conceptos fundamentales de diseño digital y buenas prácticas en VHDL orientado al diseño automático de sistemas fiables.